Project Architecture (Standalone)

- Use:
  - Standalone Components (`standalone: true`)
  - Standalone Pipes & Directives
  - `bootstrapApplication()` instead of AppModule
  - `provideRouter()` for routing
- Organize folders:
  - `core/` → interceptors, guards, global services
  - `shared/` → reusable components, pipes
  - `features/` → app features (products, cart, etc.)

3. Environment Configuration

- Create:
  - `environment.ts`
  - `environment.prod.ts`
- Add `baseUrl` for API
- Use `baseUrl` in all services
- ❌ Do NOT hardcode URLs anywhere

4. Interceptors (Global HTTP Handling)

- Create **Auth Interceptor**
  - Add `Authorization: Bearer <token>` to all requests
- Create **Error Interceptor**
  - Handle:
    - 401 → redirect to login
    - 404 → show message
    - 500 → generic error
- Create **Loading Interceptor**
  - Show global spinner on request start
  - Hide spinner on response (success/error)

5. Authentication & Authorization

- Implement:
  - Sign Up
  - Sign In
  - Forgot Password
  - Reset Password
- Store token in `localStorage`
- Create **Auth Guard (Standalone)**
  - Protect private routes
  - Redirect unauthenticated users to login

6. Routing (Standalone Routing System)

- Use `provideRouter()`
- Implement lazy loading using:
  - `loadComponent`
- Apply guards to protected routes
- Organize routes per feature

7. Categories Feature

- Get all categories
- Get specific category
- Display categories in UI

8. SubCategories Feature

- Get all subcategories
- Get specific subcategory
- Get subcategories by category
- Display properly in UI

9. Brands Feature

- Get all brands
- Get specific brand
- Display brands in UI

10. Products Feature

- Get all products
- Get product by ID
- Display product list & details
- Implement filtering:
    - By category
    - By brand
- Add pagination (if supported)

11. Wishlist Feature

- Add product to wishlist
- Remove product from wishlist
- Get logged user wishlist
- Sync UI with backend state

12. Cart Feature

- Add product to cart
- Update product quantity
- Remove specific item
- Clear entire cart
- Get logged user cart
- Reflect changes instantly in UI

13. Orders Feature

- Create Cash Order
- Implement Checkout Session (online payment)
- Get all orders
- Get logged user orders
- Show order history in UI

14. User Profile Feature

- Update user data
- Change password
- Manage addresses:
    - Add address
    - Remove address
    - Get user addresses

15. Payment Integration

- Support:
    - Cash payment
    - Online payment (simulation or real)
- Build checkout page
- Handle:
    - Payment success
    - Payment failure

16. Chatbot Feature

- Build chat UI component
- Implement FAQ-based responses
- (Bonus) Integrate with external service like Dialogflow

17. UI / UX Requirements

- Responsive design (mobile + desktop)
- Global loading spinner
- Clear error messages
- Toast notifications for user feedback

18. Code Standards

- Use services for API communication
- Use interfaces/models for typing
- Separate business logic from UI
- Write clean, maintainable code
- Reuse components where possible